

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Мухамедьянова А.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 16.

Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода.

Вариант 16) Правило проверки: строка должна оканчиваться на «.» или «;»

Общий метод и алгоритм решения

Кратко опишите системные вызовы, которые вы использовали в лабораторной работе.

Использованные системные вызовы:

- `pid_t fork(void);` — создает новый (дочерний) процесс, копируя контекст вызывающего процесса.
- `int pipe(int *fd);` — создает однонаправленный канал межпроцессного взаимодействия с двумя файловыми дескрипторами: `fd[0]` для чтения и `fd[1]` для записи.
- `int open(const char *pathname, int flags, mode_t mode);` — открывает или создает файл для записи результатов работы программы с заданными правами доступа.
- `int dup2(int oldfd, int newfd);` — дублирует файловый дескриптор `oldfd` в `newfd`, позволяя перенаправить стандартные потоки ввода/вывода (`stdin`, `stdout`) и пользовательский дескриптор ошибок.
- `int execve(const char *filename, char *const argv[], char *const env[]);` — загружает и исполняет новую программу (`./child`) в контексте текущего дочернего процесса.
- `ssize_t read(int fd, void *buf, size_t count);` — выполняет чтение данных из файлового дескриптора в буфер памяти.
- `ssize_t write(int fd, const void *buf, size_t count);` — записывает данные из буфера памяти в файловый дескриптор.
- `int close(int fd);` — закрывает файловый дескриптор, высвобождая ресурсы и сигнализируя о закрытии потока (в том числе для отправки EOF в пайп).
- `pid_t waitpid(pid_t pid, int *wstatus, int options);` — приостанавливает выполнение родительского процесса до момента завершения дочернего процесса.

Программа состоит из двух независимых исполняемых файлов: родительского (`parent.c`) и дочернего (`child.c`). Взаимодействие между ними организовано через два канала (`pipe`):

`pipe1`: канал для передачи входных строк от родительского процесса в дочерний.

`pipe2`: канал для передачи сообщений об ошибках из дочернего процесса обратно в родительский.

1) инициализация родительского процесса (`parent.c`):

- создаются два межпроцессных канала `pipe1` и `pipe2`.

- считывается имя файла, после чего он открывается системным вызовом `open()` с флагами `O_WRONLY | O_CREAT | O_TRUNC` (запись, создание при отсутствии, очистка при существовании).
- посредством вызова `fork()` порождается дочерний процесс.

2) настройка дочернего процесса:

- в ветке дочернего процесса (`pid == 0`) выполняется перенаправление файловых дескрипторов вызовом `dup2()`:
 - Входной конец `pipe1[0]` перенаправляется на `stdin (0)`.
 - Дескриптор открытого файла `file_fd` перенаправляется на `stdout (1)`.
 - Выходной конец `pipe2[1]` перенаправляется на дескриптор `10 (ERROR_FD)`.
- неиспользуемые концевые дескрипторы каналов закрываются вызовом `close()`.
- вызовом `execve()` запускается бинарный файл `./child`.

3) обработка данных в дочерней программе (`child.c`):

- программа считывает входящий поток данных из `stdin` (поступающий из `pipe1`) в буфер.
- поток разбирается на отдельные строки (разделитель `\n`).
- каждая строка проверяется на соответствие: символом перед переносом строки должна быть точка `.` или точка с запятой `;`.
- при успешной проверке: строка записывается в `stdout`, благодаря чему сразу попадает в выходной файл.
- при несоответствии правилу: в дескриптор `ERROR_FD (10)` отправляется сообщение "Error: string must end with '.', or ';\n", поступающее в канал `pipe2`.

4) завершение и вывод ошибок:

- родительский процесс считывает строки пользователя из `stdin` и отправляет их в `pipe1[1]`.
- по завершении ввода родитель закрывает запись `pipe1[1]`, оповещая дочерний процесс об окончании передачи.
- родитель вычитывает из `pipe2[0]` сообщения об ошибках от дочернего процесса и выводит их в консоль (`stdout`).
- вызовом `waitpid()` родитель дожидается полного завершения дочернего процесса и завершает работу.

Код программы

parent.c

```
#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <fcntl.h>

#include <stdlib.h>

#define BUFFER_SIZE 1024

#define ERROR_FD 10

int main()
{
    int pipe1[2];
    int pipe2[2];

    if(pipe(pipe1) == -1){
        write(2, "pipe1 error\n", 12);
        exit(1);
    }
    if(pipe(pipe2) == -1){
        write(2, "pipe2 error\n", 12);
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    int filename_len = read(0, buffer, BUFFER_SIZE - 1);
    if (filename_len <= 0){
        write(2, "filename read error\n", 20);
        exit(1);
    }
    if (buffer[filename_len-1] == '\n'){
```

```
    buffer[filename_len-1] = '\0';
}
else {
    buffer[filename_len] = '\0';
}

int file_fd = open(buffer, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (file_fd == -1){
    write(2, "file open error\n", 16);
    exit(1);
}

pid_t pid = fork();
if (pid == -1){
    write(2, "fork error\n", 11);
    close(file_fd);
    exit(1);
}

if (pid == 0){
    close(pipe1[1]);
    close(pipe2[0]);

    if(dup2(pipe1[0], 0) == -1){
        write(2, "dup2 stdin error\n", 17);
        exit(1);
    }

    if(dup2(file_fd, 1) == -1){
        write(2, "dup2 stdout error\n", 18);
        exit(1);
    }

    if(dup2(pipe2[1], ERROR_FD) == -1){
```

```
    write(2, "dup2 error fd error\n", 20);  
    exit(1);  
}
```

```
close(pipe1[0]);  
close(file_fd);  
close(pipe2[1]);
```

```
char *args[] = {  
    (char *)"./child",  
    NULL  
};
```

```
char *env[] = {  
    NULL  
};
```

```
execve("./child", args, env);  
write(2, "exec error\n", 11);  
exit(1);  
}
```

```
close(pipe1[0]);  
close(pipe2[1]);  
close(file_fd);
```

```
while(1){  
    int b_read = read(0, buffer, BUFFER_SIZE);  
    if(b_read == 0){  
        break;  
    }  
}
```

```
if(b_read < 0){  
    write(2, "read error\n", 11);  
    break;  
}
```

```
int b_written = 0;
```

```
while(b_written < b_read){  
    int res = write(pipe1[1], buffer + b_written, b_read - b_written);  
    if (res < 0){  
        write(2, "pipe1 write error\n", 17);  
        break;  
    }  
    b_written += res;  
}  
}
```

```
close(pipe1[1]);
```

```
while(1){  
    int b_read = read(pipe2[0], buffer, BUFFER_SIZE);  
    if(b_read == 0){  
        break;  
    }  
    if(b_read < 0){  
        write(2, "pipe2 read error\n", 17);  
        break;  
    }  
    write(1, buffer, b_read);  
}
```

```
close(pipe2[0]);
```

```
waitpid(pid, NULL, 0);  
return 0;  
}
```

child.c

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#define BUFFER_SIZE 1024
```

```
#define ERROR_FD 10
```

```
int main(){  
    char buffer[BUFFER_SIZE];  
    while(1){  
        int b_read = read(0, buffer, BUFFER_SIZE);  
        if(b_read == 0){  
            break;  
        }  
        if(b_read < 0){  
            write(2, "read error\n", 11);  
            exit(1);  
        }  
  
        int start = 0;  
        while(start < b_read){  
            int end = start;  
            while(end < b_read && buffer[end] != '\n'){  
                end++;  
            }  
            int last = end - 1;  
            if(last >= start && (buffer[last] == '.' || buffer[last] == ';')){  
                int length = end - start;  
                int b_written = 0;
```

```

while(b_written < length){
    int res = write(1, buffer + start + b_written, length - b_written);
    if(res < 0){
        write(2, "file write error\n", 17);
        exit(1);
    }
    b_written += res;
}
if(end < b_read && buffer[end] == '\n'){
    write(1, "\n", 1);
}
}
else{
    char error_mes[] = "Error: string must end with '.', or ';\n";
    write(ERROR_FD, error_mes, sizeof(error_mes)-1);
}
if (end < b_read && buffer[end] == '\n'){
    start = end + 1;
}
else{
    start = end;
}
}
}
return 0;
}

```

Протокол работы программы

Тестирование:

Script started on 2026-10-05 21:52:01+03:00 [TERM="xterm-256color" TTY="/dev/pts/4"
COLUMNS="170" LINES="9"]

```
allie@parsley:~$ cc parent.c -o parent
```

```
allie@parsley:~$ cc child.c -o child
```

```
allie@parsley:~$ ./parent
```

```
result.txt
```

```
moloko
```

```
hleb;
```

```
drozhzhi;
```

```
yabloki!
```

```
pups&
```

```
paketi.
```

```
Error: string must end with '.', or ';'

```

```
Error: string must end with '.', or ';'

```

```
Error: string must end with '.', or ';'

```

```
allie@parsley:~$ cat result.txt
```

```
hleb;
```

```
drozhzhi;
```

```
paketi.
```

```
allie@parsley:~$ exit
```

```
exit
```

```
Script done on 2026-10-05 21:54:39+03:00 [COMMAND_EXIT_CODE="0"]
```

Вывод

В ходе лабораторной работы была успешно реализована программа на языке C для межпроцессного взаимодействия с использованием системных вызовов `fork`, `pipe`, `read`, `write`, `open`, `close`, `dup2` и `execve`. Родительский процесс считывает данные пользователя, передает их дочерней программе через канал и выводит полученные сообщения об ошибках. Дочерний процесс проверяет строки на соответствие заданному правилу (окончание на «.» или «;»), сохраняя валидные строки в файл. Все поставленные в работе задачи были выполнены в полном объеме.